

Document Management

Week 4 validation report

STATUS: COMPLETE

Policy storage, retrieval, metadata, and version history

Week 4 focused on proving that DeRexi can manage policy documents as structured records. The policy library is stored in Supabase, exposed through the FastAPI backend, and includes ownership, category, review status, and version information.

01

Store policies

Policies and metadata are present in the database.

02

Retrieve policies

FastAPI can list and open individual policy records.

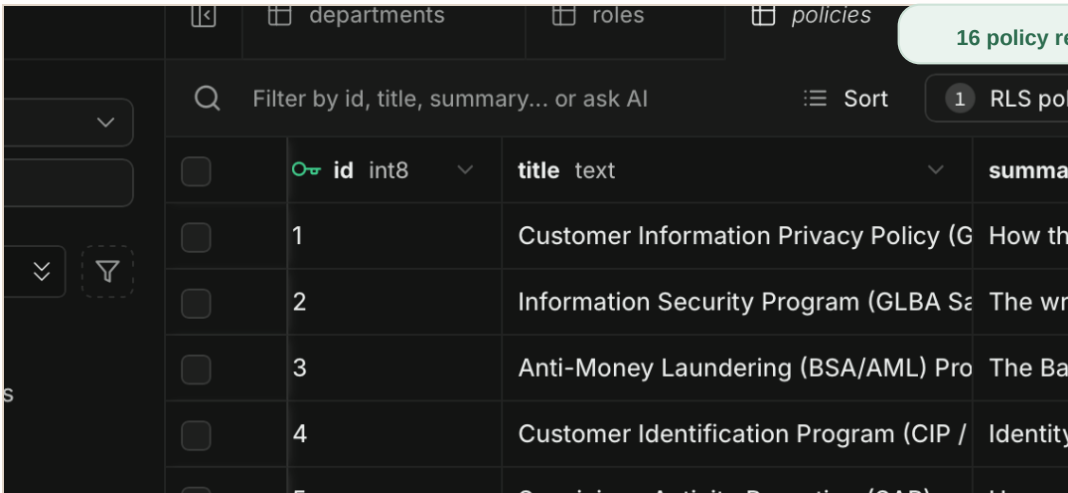
03

Track versions

Multiple versions can be retained under one policy.

DATABASE EVIDENCE

Policy library stored in Supabase



The screenshot shows the Supabase web interface for the 'policies' table. A green badge in the top right corner indicates '16 policy records'. The table has columns for 'id' (int8), 'title' (text), and 'summary' (text). The first four rows are visible:

	id	title	summary
1	1	Customer Information Privacy Policy (G	How th
2	2	Information Security Program (GLBA Se	The wr
3	3	Anti-Money Laundering (BSA/AML) Pro	The Ba
4	4	Customer Identification Program (CIP /	Identity

API RETRIEVAL PROOF

```
GET /policies/11
title: "Wealth Management & Private Banking Suitability"
latest_version: "2.0"
review_date: "2026-10-01"
versions retained: 2 (1.0 and 2.0)
```

Result: DeRexi can store, retrieve, and version policy records successfully.

Quality Fix

Always resolve the newest policy version

A small retrieval rule was added after testing uncovered a tied-timestamp edge case.

WHAT HAPPENED

Policy versions 1.0 and 2.0 were created at the exact same timestamp. Sorting by time alone allowed the older version to appear as current.

WHAT CHANGED

DeRexi now sorts by newest timestamp first, then uses the higher record ID as a tie-breaker. This keeps the latest policy version consistent.

THE NEW RETRIEVAL RULE

- 1. newest created_at -> 2. if tied, highest id -> current version

Applied to policy details, policy search / ask, and the policy-health dashboard.

OPENCODE VERIFICATION

```
102 - Copied the DeRexi NS + CT+V assignment reports into `derexi-policy-pilot/
reports/` ('network-security/', 'cyber-threats-and-vulnerabilities/');
untouched
103 +
104 + ### September 21, 2026 - Deterministic "current version" resolution
105 + - Found seeded edge case: policy 11 had v1.0 (id 11) and v2.0 (id 12)
    + **identical `created_at`** timestamps; `policy_to_dict` ordered by `created_at`
    + only, so `GET /policies/11` nondeterministically reported `latest_version`
106 + - Fixed with secondary key `id DESC` in `policy_to_dict` (verified `GET /policies/11`
    + → `latest_version 2.0`, `review_date 2026-10-01`)
107 + - Same tiebug existed in `SEARCH_SQL` (`/policies/search`, `/ask`) and
    + `reviews_due` CTE; Kiona approved applying the same tiebreaker → both
    + `pv.created_at DESC, pv.id DESC`
108 + - Verified via direct query: current-version CTE picks v2.0 (id 12) w/
    + vs v1.0 (id 11) without; dashboard + search endpoints still green; `policy_health`
109 + - No schema changes

+ Thought: 1.3s

Done. Three changes in derexi-policy-pilot/backend/main.py (no schema changes,
endpoints):

1. policy_to_dict - ORDER BY created_at DESC, id DESC
2. SEARCH_SQL (feeds /policies/search + /ask) - ORDER BY pv.policy_id, pv.created_at DESC, pv.id DESC
3. Dashboard reviews_due CTE - ORDER BY pv.policy_id, pv.created_at DESC, pv.id DESC
```

BEFORE / AFTER

Before

latest_version: "1.0"
review_date: "2025-06-01"
The tie could select the older version.

After

latest_version: "2.0"
review_date: "2026-10-01"
The newest version is selected consistently.

Week 4 takeaway: policy content is versioned, retrievable, and protected against stale-version answers.